Exam Day Playbook — COMS3011A

Monday 2 November 2026 · Lab PCs (Ubuntu, Node 18.19.1) · Qoder · 08:00–17:00

The big idea: the AI is great at *doing* and bad at *guessing*. Your job today is to **plan, direct and check**, not to type code. You'll do one small task at a time, each in a fresh Qoder chat, check it yourself, then commit it. That's the whole method.

The day at a glance

Time	What you do
Before 08:00	Log in, get the files, set up git, sign in to Qoder (Step 1)
08:00–08:30	Planning: no coding. Read the brief, run the Master Plan Prompt (Step 2)
08:30–09:00	Create the project and the first commit + push (Step 3)
09:00–16:00	The task loop: one task per chat, check, commit (Step 4). Push every hour (Step 5)
~12:30–13:30	Lunch. Push before you go; walk through the app when you're back
16:00	Feature freeze. Only fixes from now on (Step 7)
16:45	Done. Final push, check GitHub, log out of git and Qoder

Qoder in 60 seconds

Qoder is a code editor (it looks just like VS Code) with an AI helper in a chat panel on the right. Use the **Qoder app**, not the CLI. A "project" is just a **folder**: open it with **File** → **Open Folder**. The AI can only see files inside that folder.

Thing	What it means	When
Ctrl+L	Open the chat panel	
Ctrl+`	Open the terminal (bottom)	
New chat (+)	Fresh memory for the AI	Every new task
Ask mode	AI can only talk	Planning, questions, reviews
Agent mode	AI edits files and runs commands	Building and fixing
Model: Auto	Good default, cheap	Almost everything
Model: Ultimate	Smartest, about 4x the credits	The master plan; bugs Auto failed twice
@file	Attach a file to your message	Always: @PLAN.md @AGENTS.md
Ctrl+Enter / Ctrl+Backspace	Accept / reject all the AI's changes	After reading the diff
Ctrl+Shift+V	Paste in the terminal	Your GitHub token, commands

Step 1 — Before 08:00: get set up (15 min)

1. Log in to the lab PC. Open a terminal (search for **Terminal** in the apps menu).
2. Check the machine:

```
node -v # expect v18.19.1
npm -v
git --version
python3 --version
```

3. Get your files (they're public on GitHub):

```
cd ~
git clone https://github.com/Suhai1/coms3011a-test-practice.git pack
```

If that fails: download them from the Moodle upload, or read them on your phone.

4. Tell git who you are, and let it remember your token for the day:

```
git config --global user.name "Suhail Seedat"
git config --global user.email "<your GitHub email>"
git config --global credential.helper 'cache --timeout=36000'
```

5. Open **Qoder** and sign in. Don't open any folder yet.
6. Have your **GitHub token** ready on your phone (starts with `ghp_`). It's your "password" for the first push.

Step 2 — 08:00–08:30: plan (no coding allowed)

1. **Do nothing for 2 minutes.** Breathe. You have plenty of time.
2. Read the brief **twice**. Note every "must", "required" and "done when".
3. Check the allowed-domains list. If npm isn't on it, tell the invigilator immediately.
4. In Qoder: **new chat** → **Ask mode** → **Ultimate**. Open `pack/PLAN-PROMPT.md`, copy the **Master Plan Prompt** (shown in full on the last pages of this PDF), and paste the **entire brief at the end**. Send.
5. Read the plan it gives you. Push back where it's wrong: "the brief says X", "task 9 should come before task 6". Keep going until it matches the brief.
6. Check the plan has: the marks map, the hard rules, a task list with **Done when** checks, and the **MINIMUM PASSING VERSION** line.

Asking Qoder questions in Ask mode is planning, not coding. Don't let it create or edit files before 08:30.

Step 3 — 08:30: create the project (15–20 min)

Run these in the terminal **exactly**. The pinned versions are what make Node 18 work (tested on 18.19.1):

```
cd ~
npx -y create-next-app@15.5.27 exam-app --ts --tailwind --eslint --app --src-dir --use-npm
--import-alias "@/*" --turbopack --yes
cd exam-app
npm install -E -D tailwindcss@4.1.18 @tailwindcss/postcss@4.1.18
npm install -E better-sqlite3@11.10.0
npm install -E -D @types/better-sqlite3 vitest@3.2.4 tsx
mkdir -p data src/lib src/tests scripts
npm pkg set scripts.test="DB_PATH=:memory: vitest run"
npm pkg set scripts.seed="tsx scripts/seed.ts"
cp ~/pack/AGENTS.md ~/pack/COMMITS.md ~/pack/DESIGN.md .
npm run build
```

- `npm run build` must finish with a list of routes. If it complains about **tailwindcss-oxide**, rerun the Tailwind line.
- If **better-sqlite3** fails to install: use sql.js instead (`STACK-A-NEXTJS.md`, Option 2). Don't switch stacks for this.

- In Qoder: **File** → **Open Folder** → **exam-app**. Agent mode: "Save the final plan from our planning chat exactly as written to PLAN.md". If the planning chat is in another window, copy-paste the plan into a new file `PLAN.md` yourself. Also save the brief as `BRIEF.md`.
- Fill in section 8 of `AGENTS.md` (app name, rules from the brief).

Now make an **empty** repo on github.com (no README), then:

```
printf "data/*.db\ndata/*.db-*\n.env\n" >> .gitignore
git init
git add .
git commit -m "chore: scaffold project with plan"
git branch -M main
git remote add origin https://github.com/Suhai1/<repo-name>.git
git push -u origin main
```

Username = `Suhai1`. Password = **your token** (paste with Ctrl+Shift+V; nothing shows while you paste; that's normal). Refresh GitHub and check your files are there.

Step 4 — The task loop (repeat all day)

This is what you did in the test (a plan, then a fresh chat per step), plus a **check** and a **commit** after every task.

#	Do this
1	**New chat** (+). Agent mode. Model **Auto**.
2	Paste the **Task Prompt** (below) with the task number.
3	Qoder restates the task and lists files. Wrong? Correct it. Right? Say **OK**.
4	Qoder builds. When it asks to run a command, **read it first**. Approve `npm test` / `npm run build`. **Reject** `npm update`, `npm audit fix --force`, `@latest`, `rm -rf`.
5	Look at the changed files. Unexpected files changed? Any `DELETE FROM`? Reject and say so.
6	**Check it yourself:** `npm run dev` → http://localhost:3000 → do the task's **Done when** check. Never trust "it works!"
7	Works → tick it in PLAN.md → `git add . && git commit -m "feat(scope): what it does"`
8	Broken → describe exactly what's wrong (see Step 6). Then back to 6.

Task Prompt (copy, change [N])

```
@PLAN.md @AGENTS.md @DESIGN.md
Do Task [N] from PLAN.md, and only that task.

1. First, restate the task's Goal, Rules to respect and Done-when in 3 lines, and list the files
you'll change. Wait for my OK.
2. Then implement it. Logic and SQL in src/lib, thin routes/pages, UI following DESIGN.md.
3. Add the tests listed for this task. Run `npm run build` and `npm test` and show me the results
honestly.
4. Tell me exactly how to check the Done-when myself (what to click / what to run).
5. When I confirm it works, tick Task [N] in PLAN.md as [x].
Do not start the next task. Do not change unrelated code. Do not install packages without asking
(Node 18, pinned versions).
```

When the plan says VERIFY

```
@PLAN.md @AGENTS.md
VERIFY checkpoint. Don't add features.
1. Run npm run build and npm test; report results.
2. List every completed task's Done-when check from PLAN.md so I can click through them all now.
3. Review the code changed since the last checkpoint against AGENTS.md section 3 (database rules)
and list any violations.
Fix only real breakages, one at a time, with my OK.
```

Step 5 — Every hour, on the hour (phone alarms 10:00–16:00)

1. `git push`
2. Click through the **whole app** like a user, from start to finish.
3. `npm test`
4. Anything broken → fix it **before** the next task.
5. Behind schedule? Use the **Re-plan prompt** (below).

```
@PLAN.md
It's [time]. Tasks [x, y] are done. I'm behind / [what changed].
Re-plan the remaining tasks to finish by 16:00. Keep all core tasks. Drop or shrink optional
tasks with the lowest marks ÷ effort, and tell me what you dropped. Update PLAN.md.
```

Step 6 — When things go wrong

Problem	Do this
Something's broken	Tell Qoder **exactly**: what happened, what you expected, the steps, the error text. "It's broken" gets you a random guess.
It fails the same fix twice	"Stop. Explain the root cause and two different approaches before changing anything." Switch the model to **Ultimate** for this.
It's going in circles (10+ min)	`git stash` (puts the mess aside), **new chat**, ask again with what you learned.
It changed lots of unrelated files	Reject all (Ctrl+Backspace). "Only change X in @file."
Chat getting long or the AI getting dumber	New chat. Attach @PLAN.md @AGENTS.md again.
Install error / Node 18 error	See `STACK-A-NEXTJS.md` → "Node 18 gotchas".
Want to undo a pushed commit	`git revert <hash>` then `git push`

Every 3–4 tasks, open a **new chat in Ask mode** and paste **Prompt 5 (critic review)** from `PROMPTS.md`. A fresh reviewer catches what the builder missed.

Step 7 — Finish (16:00–17:00)

- **16:00 — feature freeze.** No new features. Half-finished feature? Hide it or remove it: broken things lose marks in a live demo.
- Paste **Prompt 10 (final checklist)** from `PROMPTS.md`: build, tests, README with run commands, cleanup.
- **16:30** — full walkthrough of every Done-when check, then `npm test`, then `git push`.
- Open your repo on **github.com** and confirm the latest commit is really there.
- **16:45** — stop. Nothing in flight.
- Before you leave: `git credential-cache exit` (logs your token out) and sign out of Qoder.

The Master Plan Prompt

08:00, **Ask mode, Ultimate**. Paste all of this, then the **full brief** after the last line.

```
You are my senior engineer and exam strategist. I have a 7.5-hour practical software exam,
building an application with you, an AI coding agent. It is marked LIVE by a demonstrator at the
end, against the brief's requirements and "done when" conditions. Your job right now is ONLY to
plan. Do NOT write application code, and do not create or edit files.

## My environment (fixed: plan around it)
- Ubuntu lab PC, Node 18.19.1, firewalled internet (npm registry and docs are reachable; assume
CDNs, Google Fonts and most other sites are blocked).
- Stack: Next.js 15.5.27 (App Router) + TypeScript (strict) + Tailwind 4.1.18 (pinned) + SQLite
via better-sqlite3 11.10.0 (fallback: sql.js), Vitest 3.2.4 for tests. Raw SQL with ?
placeholders, no ORM.
- Every package must support Node 18. No external services, no auth providers, no CDN scripts.
- Business logic lives in src/lib/<resource>.ts; API routes and pages are thin and call lib
functions.
- My full rules are in AGENTS.md (I'll attach it during the build). Key ones: never delete rows
(archive with archived_at), never store derived values, fixed enums enforced with CHECK
constraints, versioned migrations (never drop/recreate), index filtered and sorted columns, no
N+1 queries, cycle guards on any graph/tree, tests against an in-memory database (never mocked).

## How I will work
- One task per fresh chat. Each task must be finished, verified in the browser, tested and
committed before the next starts.
- I will paste a task prompt like: "@PLAN.md @AGENTS.md Do task N only."
- So every task must be self-contained, small (10–25 minutes of agent work), and checkable by me
in under 60 seconds.

## Produce the plan in EXACTLY this structure (markdown):

### 1. The brief in one paragraph
What the app is, who uses it, and how it will be marked.

### 2. Marks map
A table: requirement / feature | marks or weight (quote the brief) | required or optional | "done
when" condition quoted word-for-word from the brief. If the brief gives no marks, rank by
importance.

### 3. Hard rules and traps
Every constraint in the brief that an AI agent is likely to break by accident (e.g. deletion,
stored derived values, fixed lists, time zones, persistence across restart). One line each,
quoting the brief.

### 4. Data model
Complete SQLite CREATE TABLE and CREATE INDEX statements for migration 1, with a one-line reason
for each table. Include archived_at, created_at, updated_at, CHECK constraints for fixed enums,
foreign keys, and join tables for many-to-many.

### 5. Screens and API
- Pages/views: route, what's on it, which tasks build it.
- API routes: method + path, input, output, error cases.

### 6. Task list (the most important part)
Numbered tasks in build order. Rules:
- Task 1 = foundation: db.ts, migrations, design tokens from DESIGN.md, layout/nav, one passing
test. No features.
- Then ALL required/core features, one per task, smallest working version first.
- Then optional features in order of (marks ÷ effort), highest first.
- Insert a "VERIFY" task after every 3–4 build tasks: run all tests, click through the full user
journey, fix regressions only.
```

```
- Put a clear line "---- MINIMUM PASSING VERSION ENDS HERE ----" after the last core task.
- Final tasks: seed data with realistic content, design pass against DESIGN.md, README with run
instructions, final check.

For EACH task use this exact format:
**Task N — <name>** (est. <minutes> min, <marks> marks, depends on: <task numbers or none>)
- Goal: <one sentence, user-visible outcome>
- Files: <files to create or change>
- Rules to respect: <the specific AGENTS.md / brief rules this task could break>
- Done when: <a concrete check I can do in the browser or terminal in under 60 seconds, mirroring
the brief's wording where possible>
- Tests: <the behaviour tests to add, including the awkward case>

### 7. Time budget
Map the tasks onto the day: 08:30 start, 1-hour lunch around 12:30–13:30 (assume this unless the
brief says otherwise), feature freeze at 16:00, finished by 16:45. Mark where I should be by
10:30, 12:30 and 15:00. If the task list can't fit, say which optional tasks to drop.

### 8. Risks and ambiguities
- Anything unclear or contradictory in the brief, with the interpretation you recommend and why
(I may ask the lecturer).
- Anything that may not work on Node 18 or behind the firewall, with a fallback.

Be specific to THIS brief. No generic advice. If something isn't in the brief, don't invent
requirements; list it under ambiguities instead.

=== PASTE THE FULL BRIEF BELOW THIS LINE ===
```